

Installation Documentation

Project: Smart Tracking System

Group: Group 4 - Container and Container Management

1. Prerequisites

- **Docker Desktop** must be installed and running in the background on your operating system (the whale icon should be green).
- You must have command-line access via Terminal (Mac/Linux) or Command Prompt (Windows).
- The project source codes and the docker-compose.yml file must be downloaded to the computer.

2. Compilation and Deployment with Standard Docker (Optional/Testing Phase)

Before moving to Swarm orchestration, it is recommended to build the microservice images from the source codes and test them in a standard Docker environment.

2.1. Building Images

Open the terminal, navigate to the main project directory (the folder containing the docker-compose.yml file), and run the following command to build all service images:

```
docker compose build
```

2.2. Starting with Standard Compose

The following command is used to bring the system up as standalone containers:

```
docker compose up -d
```

The operation of the system can be verified using the docker ps command.

3. Docker Cleanup (Preparation for Swarm)

When transitioning to Swarm orchestration, which is the main project of Group 4, all containers running on standard Docker and residual networks must be cleaned up to prevent port and network conflicts. Run the following commands sequentially:

```
# 1. Varsa eski compose ağlarını ve servislerini kapatır
```

```
docker compose down
```

```
# 2. Arka planda açık kalmış tüm bağımsız konteynerleri zorla siler
```

```
docker rm -f $(docker ps -aq)
```

```
# 3. Kullanılmayan ve çakışma yaratan tüm eski ağları sistemden kazır
```

```
docker network prune -f
```

4. Installation with Docker Swarm (Orchestration)

These are the steps to transition to the Container Management phase, which is the heart of the project.

4.1. Activating Swarm Mode

To convert the current computer into a Swarm Manager Node, the following command is run in the terminal:

docker-compose.yml dosyasında eğer swarma geçişte aşağıdaki dönüşümü yap:

swarm olmadan ->

networks:

```
  microservice_network:  
    driver: bridge
```

swarm olduğunda->

networks:

```
  microservice_network:  
    driver: overlay
```

Neden bunu yaptık?

Ben bu `akilli_takip_microservice_network` adında bir ağ buldum ama bu bir Swarm (overlay) ağı değil, eski usül standart bir ağ. Ben Swarm servislerini bu eski ağa bağlayamam!"

1. Bilgisayarını Swarm Yöneticisi yapar

```
docker swarm init
```

Note: The message "Swarm initialized: current node is now a manager" should be seen on the screen.

4.2. Live Deployment with Stack Deploy

A single deployment command is used to bring the databases, message queue (RabbitMQ), and all microservices live under Swarm orchestration with a dedicated overlay network:

2. Tüm sistemi (Stack) Swarm ağı üzerinde canlıya alır

```
docker stack deploy -c docker-compose.yml akilli_takip
```

5. Verification and Testing Steps

5.1. Checking Service Statuses

To confirm that the Swarm manager has started all services without errors:

3. Servislerin ayağa kalkışını izlemeni sağlar (Hepsi 1/1 olana kadar ara ara yaz)

```
docker service ls
```

*Note: When the command is run, it should be seen that **1/1** is written in the **REPLICAS** column for all services in the table. It may take 10-20 seconds for the database and queue services to become fully ready.*

5.2. Interface and System Tests

- **Dashboard (Front End):** The operation of the cargo management interface is verified by navigating to `http://localhost:3000` via the browser.
- **RabbitMQ (Message Queue):** The asynchronous messaging panel between microservices is accessed by navigating to `http://localhost:15672` (Username: guest, Password: guest).

6. System Shutdown and Cleanup (Optional)

To completely stop the project, exit the Swarm environment, and return the system to its initial state, the following commands are run sequentially:

```
# 1. Swarm üzerindeki "akilli_takip" sistemini tüm servisleriyle birlikte siler
```

```
docker stack rm akilli_takip
```

```
# Ağların tamamen kopması için burada 10 saniye bekle, ardından şunu yaz:
```

```
# 2. Swarm yöneticiliğinden istifa eder ve Swarm modunu tamamen kapatır
```

```
docker swarm leave --force
```